

Polyvoice macro-cell cookbook

Worked examples for the five macro verbs (`drumkit`, `kit`, `chord`, `yarns`, and the deferred `mutes/veils`). The grammar spec is on Marginalia #189 note #224; the implementation log is across recent `purperl-tidal` + `fh2-config` commits.

This is a usage doc, not a design doc — copy the examples into a Calypso cell, fire, observe.

Quick reference

Verb	Allocates FH-2 outputs?	Names voices?	Dispatch shape
<code>drumkit</code>	Yes (gate + pitch × N)	Yes (per-voice names)	One MIDI note per pattern token, fixed routing
<code>kit</code>	No (uses <code>drumkit</code> 's)	No (uses tokens as binding names)	Each token looks up a binding and dispatches it
<code>chord</code>	Yes (gate + pitch × N)	Synthesised only	Broadcast each note to all N voices with interval offsets
<code>yarns</code>	Yes (gate + pitch × N)	Synthesised only	Allocate each note to one voice (round-robin / mono / unison)
<code>mutes</code>	No (controller-side)	—	Deferred — see #225
<code>veils</code>	No (controller-side)	—	Deferred — see #225

`drumkit`, `chord`, `yarns` all claim FH-2 hardware through the `apply-<verb>` daemon line and tag ownership with their own `OwnerKind` in the `ClaimRig` (`OwnDrumKit` / `OwnChord` / `OwnYarns`).

A practical note: `<>` line continuation

Multi-line cells need a trailing `<>` on every line except the last, so Calypso bundles them into one wire frame. Single-line cells don't need any markers. This is the whole language convention, not specific to these verbs.

```
drumkit kitA [bd sn hh cp] <>
  gates gt0 <>
  pitch main <>
  ch 10
```

vs single-line:

```
drumkit kitA [bd sn hh cp] gates gt0 pitch main ch 10
```

Both work. Calypso's collapser already knows about all six macro verbs (committed in calypso [c188092](#)).

drumkit — named voices, fixed routing

The classical drum-machine shape. N named voices on consecutive MIDI channels, each on its own FH-2 gate + pitch CV.

Auto-allocated slots (preferred)

```
drumkit kitA [bd sn hh cp] <>
  gates gt0 <>
  pitch main <>
  ch 10
```

- 4 voices: **bd**, **sn**, **hh**, **cp**
- Gates auto-allocate slots 0..3 of bank **gt0**
- Pitch CVs auto-allocate slots 0..3 of bank **main**
- Channels: bd=10, sn=11, hh=12, cp=13

Explicit slot ranges

```
drumkit kitB [bd sn hh cp] <>
  gates gt0 4-7 <>
  pitch main 4-7 <>
  ch 11
```

For a kit on outputs 5–8 (1-indexed panel) / slots 4–7 (0-indexed config). Channels: bd=11, sn=12, hh=13, cp=14.

Per-voice offsets (escape hatch, non-contiguous)

```
drumkit kitC <>
  gates gt0 0-6 <>
  pitch main 0-3 <>
  ch 12 <>
  <> <>
  bd: 0 <>
  sn: 2 <>
  hh: 4 <>
  cp: 6
```

When you want bd on gate slot 0, sn on slot 2, hh on slot 4, cp on slot 6. The legacy form, kept as an escape hatch. `<>` between header and body is just a visual separator and is filtered out by the parser.

Firing patterns against a drumkit

Once the kit is installed, the per-voice names exist as bindings. Use them directly in pattern cells:

```
bd "x ~ x ~"
sn "~ x ~ x"
hh "x*8"
```

Each voice has its own pattern. Composable with all the standard mini-notation: `bd "x*4"`, `sn "x(3,8)"`, `hh "x*16"`.

kit — multi-voice dispatch through one card

The single-card abstraction. Replaces N per-voice cell pairs (`bd "x ~ x ~"`; `sn "~ x ~ x"`) with one rhythmic line.

Literal mini-notation

```
kit kitA "bd sn bd cp"
```

Each token (`bd`, `sn`, `cp`) is looked up in the binding registry (populated by the drumkit cell) and that voice's MIDI-note PrimAction fires. Composes naturally:

```
kit kitA "bd*4 sn*2 cp"
kit kitA "[bd sn] [hh hh] [bd cp] hh"
kit kitA "<bd sn cp hh>*16"
```

Host-language expression form

```
kit kitA :every 4 (fast 2) "bd sn bd cp"
kit kitA :rev "bd sn hh cp"
kit kitA :jux (fast 3) "bd sn"
```

The `:expr` form runs the body through `parseEvalPattern`, the same machinery that handles `bd :rev "x*4"` on bare bindings. Combinators (`every` / `fast` / `slow` / `rev` / `jux` / ...) transform the Pattern before dispatch, so KitDispatch sees an already-rewritten token stream.

Coexisting with per-voice cells

A drumkit installs per-voice bindings (**bd**, **sn**, etc.). Both single-voice and kit cells dispatch through those bindings, so they compose at will:

```
kit kitA "bd sn bd cp"      -- main pattern through the kit
hh "x*16"                  -- hi-hat fill on its own cell, same kit
```

The **bd** events from the kit cell and the (absent) **bd** events from the bare-binding cell don't fight — they're different cell sources both firing the same voice.

chord — broadcast with interval offsets

Each pattern token is a root note; all N voices fire in parallel with intervals from the chord shape applied.

Basic

```
chord pad1 [4] <>
  shape minor7 <>
  gates gt0 4-7 <>
  pitch main 4-7 <>
  ch 12
```

Then fire patterns at it:

```
pad1 "c4 g3 a3 e4"
```

- **c4** (MIDI 60) plays C-minor7: notes 60, 63, 67, 70 on channels 12, 13, 14, 15
- **g3** (MIDI 55) plays G-minor7: notes 55, 58, 62, 65
- etc.

Chord shapes

Look up in [Tidal.Chords.chordTable](#) for the full list. Common ones:

Shape	Intervals	Notes for C root (MIDI 60)
major	[0, 4, 7]	60, 64, 67
minor	[0, 3, 7]	60, 63, 67
major7	[0, 4, 7, 11]	60, 64, 67, 71
minor7	[0, 3, 7, 10]	60, 63, 67, 70
sus4	[0, 5, 7]	60, 65, 67
dim	[0, 3, 6]	60, 63, 66

Shape	Intervals	Notes for C root (MIDI 60)
dim7	[0, 3, 6, 9]	60, 63, 66, 69
aug	[0, 4, 8]	60, 64, 68

Voice count vs interval count

Mismatch is fine:

- **Shape has fewer intervals than voices:** voices cycle the intervals. `chord pad1 [4] shape major` produces 60, 64, 67, 60.
- **Shape has more intervals than voices:** excess is unused. `chord pad1 [3] shape major7` produces 60, 64, 67 (skipping the 7th).

Re-firing replaces

```
chord pad1 [4] shape minor7 gates gt0 4-7 pitch main 4-7 ch 12
chord pad1 [4] shape major7 gates gt0 4-7 pitch main 4-7 ch 12
```

Same name = same owner = silent replace.

yarns — polyphonic voice allocation

Each pattern note is assigned to ONE voice via the allocator. The voice count + allocation strategy decides who plays what.

Round-robin (the default)

```
yarns synth1 [4] <>
  gates gt0 4-7 <>
  pitch main 4-7 <>
  ch 11
```

Defaults: `mode=poly`, `alloc=round-robin`, `glide=0`. Pattern firing:

```
synth1 "c4 e4 g4 b4 d5 f5"
```

- Note 1 (c4) → voice 0 → ch 11 → gate slot 4
- Note 2 (e4) → voice 1 → ch 12 → gate slot 5
- Note 3 (g4) → voice 2 → ch 13 → gate slot 6
- Note 4 (b4) → voice 3 → ch 14 → gate slot 7
- Note 5 (d5) → voice 0 → ch 11 (wraps)
- Note 6 (f5) → voice 1 → ch 12

You'll see the gates cascade visibly on the FH-2 panel.

Explicit mode + alloc

```
yarns synth1 [4] <>
  mode poly <>
  alloc round-robin <>
  glide 20 <>
  gates gt0 4-7 <>
  pitch main 4-7 <>
  ch 11
```

(v1 only ships `alloc round-robin` — the other strategies error clearly at parse time with a "not implemented in v1" message. `glide` is recorded in the binding but isn't applied yet; FH-2 hardware portamento integration is planned for v2.)

Mono — all notes through voice 0

```
yarns lead1 [4] <>
  mode mono <>
  gates gt0 <>
  pitch main <>
  ch 10
```

All pattern notes fire on voice 0 (gate slot 0, channel 10). The other voices are claimed but unused — useful when you want them reserved for later mode-switching without re-allocating slots.

Unison — all voices fire in parallel

```
yarns fat1 [4] <>
  mode unison <>
  gates gt0 4-7 <>
  pitch main 4-7 <>
  ch 11
```

Each pattern note triggers all 4 voices simultaneously with the same note value. Equivalent to `chord pad1 [4] shape major` with intervals `[0,0,0,0]`, but expressed semantically. For detuned fat unison you'd need per-voice fine-tune (deferred).

Cross-verb coexistence

Same-name same-kind = silent replace. Different-kind same-name = distinct owners.

```
drumkit shared [bd sn hh cp] gates gt0 pitch main ch 10
chord shared [4] shape minor7 gates gt0 4-7 pitch main 4-7 ch 12
```

Both coexist in the ClaimRig as `OwnerId OwnDrumKit "shared"` and `OwnerId OwnChord "shared"`. If they overlap on hardware slots (e.g. both claiming `gt0 0-3`), the port-claims error names both correctly and refuses the second one. To proceed, either rename one or release the conflicting owner.

Operational gotchas

`release-claim` to free outputs without bouncing

When you want to redo a macro on the same outputs but the existing claim is in the way:

```
release-claim yarns synth1
```

Daemon replies `OK release-claim yarns synth1`. Frees the gate + pitch slots. v1 doesn't clean up the dispatcher binding registry — stale binding entries just no-op since their hardware is unclaimed.

Kinds for release-claim: `polysignal` / `tvoice` / `drumkit` / `yarns` / `chord`.

Why is there nothing on the FH-2 panel?

Before a macro cell will sound, the `fh2` MIDI device alias has to be registered (`midi-device fh2 "FH-2"`). The drumkit/chord/yarns handlers auto-register it on each successful apply, but if you fire a *pattern* cell (`bd "x*4"` or `pad1 "c4"`) before any macro cell has been applied in this session, the alias is missing and dispatch silently no-ops.

Easy fix: fire the code pane (or any drumkit/chord/yarns cell) once at session start.

`<>` confusion

The `<>` marker is **end-of-line** for line continuation. A `<>` on its own line works as a visual separator in the legacy drumkit body (the parser filters all `<>` tokens) but does NOT trigger line joining if the preceding line lacks a trailing `<>`. If you see "no binding 'gates'" or "expected `shape <shapeName>` after `[N]`" when firing a multi-line cell, the most likely cause is missing trailing `<>` on the preceding lines.

Yarns name overload

`yarns` is now both a routing-grammar device-alias verb (`yarns yarns "Yarns"`) and a macro cell verb (`yarns synth1 [4] ...`). They disambiguate by token shape: if the second token starts with `[`, it's the macro form; otherwise, the device-alias form. So your existing `yarns yarns "Yarns"` in the code pane still works, and `yarns synth1 [4] ...` in cells works alongside it.

What's not here (yet)

- **mut**es and **veil**s — controller-side macros, deferred pending MIDI-IN infrastructure. See Marginalia #189 note #225 for why and the future direction.
 - **Yarns voice-allocation strategies** other than **round-robin** — **steal-oldest** / **steal-newest** deferred. The ETS allocator state is laid out to hold per-voice last-used timestamps when those land.
 - **Yarns glide** — recorded in the binding but not slewed. FH-2 hardware portamento integration via the per-MCV-voice config is the planned path.
 - **Chord voicing** keyword (drop2 / drop3 / open) — grammar reserved but not implemented; v1 always uses close voicing (literal intervals from the shape table).
 - **Custom chord shapes** (**shape [0 4 7 11]**) — only named shapes from **Tidal.Chords.chordTable** are accepted; custom-interval-list parsing not yet wired.
-

Related docs

- Marginalia #189 (fh2-config) note #224 — the grammar spec
- Marginalia #189 note #225 — mut/veil punt rationale
- Marginalia #90 (purerl-tidal) note #222 — kit verb design
- Marginalia #90 note #223 — device-as-process architectural arc
- **docs/per-voice-refactor-plan.md** — the per-voice supervision tree the dispatcher binding registry runs on top of
- **fh2-config/docs/port-claims-design.md** — the claim-table / OwnerKind / ClaimRig architecture macro cells flow through